



Social Network

FADYA HESHAM ELORABI

Id:2205177

```

edge_index = (
    torch.tensor(
        [
            [0, 1],
            [1, 0],
            [1, 2],
            [2, 1],
            [0, 2],
            [2, 0],
            [3, 4],
            [4, 3],
            [4, 5],
            [5, 4],
            [3, 5],
            [5, 3],
            [2, 3],
            [3, 2], # one connection between a benign (2) and malicious (3)
        ],
        dtype=torch.long,
    )
    .t()
    .contiguous()
)
print(edge_index)

```

-> notice that : each edge is written twice -> this indicates (undirected)

->edge index->defines our src and dst

->pytorch helps as handle the src and dst as: the first row becomes the src and the second is the dst as it shown here when printing the VAR edge index

```

tensor([[0, 1, 1, 2, 0, 2, 3, 4, 4, 5, 3, 5, 2, 3],
        [1, 0, 2, 1, 2, 0, 4, 3, 5, 4, 5, 3, 3, 2]])

```

```

-----
x = torch.tensor(
    [
        [1.0, 0.0], # Node 0 (benign)
        [1.0, 0.0], # Node 1 (benign)
        [1.0, 0.0], # Node 2 (benign)
        [0.0, 1.0], # Node 3 (malicious)
        [0.0, 1.0], # Node 4 (malicious)
        [0.0, 1.0] # Node 5 (malicious)
    ],
    dtype=torch.float,
)

```

)

->this represent our features [we have 2 features]as:

F1->if 1->Normal

F2->if 1 -> Malicious

```
y = torch.tensor([0, 0, 0, 1, 1, 1], dtype=torch.long)
print("Node labels (y):", y)
data = Data(x=x, edge_index=edge_index, y=y)

print("\nNode features (x):\n", data.x)
print("\nEdge index (source → destination):\n", data.edge_index)
print("\nLabels (y):\n", data.y)
```

->this defines the node's label

-> this creates G with x features and edges defined in the edge_index VAR

-> the Data object now stores the entire graph structure and is what the GNN model will use for training and prediction

OUTPUT FOR EXPLAINABILITY:

Node labels (y): tensor([0, 0, 0, 1, 1, 1])

Node features (x):
tensor([[1., 0.],
 [1., 0.],
 [1., 0.],
 [0., 1.],
 [0., 1.],
 [0., 1.]])

Edge index (source → destination):
tensor([[0, 1, 1, 2, 0, 2, 3, 4, 4, 5, 3, 5, 2, 3],
 [1, 0, 2, 1, 2, 0, 4, 3, 5, 4, 5, 3, 3, 2]])

Labels (y):
tensor([0, 0, 0, 1, 1, 1])

```
class GraphSAGNet(torch.nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels):
```

```

super(GraphSAGENet, self).__init__()
self.conv1 = SAGEConv(in_channels, hidden_channels)
self.conv2 = SAGEConv(hidden_channels, out_channels)

def forward(self, x, edge_index):

    print("\n=== Forward Pass START ===")
    print("Input features (x):\n", x)
    print("Edge index:\n", edge_index)

    # First layer
    x = self.conv1(x, edge_index)
    print("\nAfter conv1 (hidden layer features):\n", x)

    x = F.relu(x)
    print("\nAfter ReLU:\n", x)

    # Second layer
    x = self.conv2(x, edge_index)
    print("\nAfter conv2 (raw class scores):\n", x)

    out = F.log_softmax(x, dim=1)
    print("\nLog-softmax (final output probabilities):\n", out)

    print("=== Forward Pass END ===\n")
    return out

```

->Here we have two layer

->Conv1 takes 2 dim features and outputs 4 dim hidden embeddings

->Conv2 takes the 4 dim embeddings and outputs 2 dim class scores

```
model = GraphSAGENet(in_channels=2, hidden_channels=4, out_channels=2)
```

Simple training loop

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

```
model.train()
```

```
for epoch in range(50):
```

```
    optimizer.zero_grad()
```

```
    out = model(data.x, data.edge_index)
```

```
    loss = F.nll_loss(out, data.y) # negative log-likelihood
```

```
    loss.backward()
```

```
    optimizer.step()
```

This creates GraphSAGE network

Optimizer ->update the weights

Model start training

50 epochs -> useful for limited training data ->training runs 50 times over the graph

Out VAR-> compute the predictions

Loss function-> compares the models' prediction and the actual label [gradient descent]

loss.backward()-> computes gradients()

optimizer.step()-> updates the weights

`model.eval()
pred = model(data.x, data.edge_index).argmax(dim=1)
print("Predicted labels:", pred.tolist()) # e.g. [0,0,`

This concludes that the model learned perfectly as this matches the actual true labels printed previously

Predicted labels: [0, 0, 0, 1, 1, 1]